MODULE 1 — Intro / Translation

- **History**: Babbage *analytical engine* (~1830) = 1st general-purpose computer; **Ada Lovelace** = author of 1st program; von Neumann/Church/Turing (1930s); Grace Hopper: machine-independent languages, linker 1952. **Mechanical calculator**: data in **registers**; simple ops (+ − × ÷); typing 11 then + eleven times = 121 (multiply by repeated add; shift accumulator = faster). *Not programmable*: adding a new fn (√) needs **mechanical redesign** → motivates stored programs.
- **Von Neumann arch.** — 5 parts: **Input device, Output device, Control Unit, ALU, Memory** (CU + ALU = CPU); *programs are data in memory*; one bus = bottleneck. Work of many people.
- **What problem does a PL solve?** (1) abstract programmer from the machine (expression); (2) help programmer *think* about problems in specific ways. Technical goal of every PL: *translate* input language into something that runs on the computer.
- **Programs as data**: compilers/interpreters take programs as input; programs sit in memory as data (Scheme: code = lists).
- **Imperative** = *how* (control flow, names, statements). Subtypes: von Neumann (C, Basic, Fortran); OO (Smalltalk, C#, Java); scripting (Perl, Bash, Awk, Python, JS). **Declarative** = *what*: describe all acceptable solutions, compiler picks how. Functional (Lisp, Scheme, Haskell, OCaml, Scala, Erlang: recursion, no side effects, expressions); Logic/constraint (Prolog, SQL, Make, ASP: backtracking). Boundaries not strict.
- **Why so many languages?** evolution; commercial/proprietary advantage; special purpose (Julia, Ada, R, SQL); preference; easy to learn / to compile; error prevention; huge libraries. **Success**: easy to learn (BASIC, Pascal, Scheme); expressive (C, Lisp, APL, Perl); easy to implement (BASIC, Forth); fast code (Fortran); backer (COBOL, Ada, VB); cheap spread (Pascal, Java). **Study PLs**: weigh tradeoffs, work around limits.

Translators

- **Translator**: systems program converting one language to another (source → target). Approaches: **Assembler**; **Pure compilation** (translate all first, then run target on input; fast run); **Interpretation** (interpreter reads source + input, produces output; flexible, slower); **Hybrid** (compile to intermediate program, **Virtual Machine** runs it; Java bytecode). Compiler/interpreter/VM are themselves runnable programs.
- **JIT**: not a distinct style, an opportunistic optimization; code compiled to machine code just before/at run time (C# → CIL → machine code). C is "compiled" but printf/scanf interpret format strings.
- Other: **linker** (partial machine-language program + library routines → executable); **preprocessor** (C macros); **source-to-source** (early C++, Kotlin, TypeScript); post-compile assembly.

```
foo.c + foo.h -> preprocessor
-> compiler -> foo.s -> assembler
-> foo.o -> linker (+ libraries)
-> executable foo   (GCC toolchain)
```

Phases of Compilation

- **Frontend** (language-dependent): scanner → parser → semantic analysis & intermediate code gen. **Backend** (machine-dependent): code improvement → target code gen → machine-specific improvement. **Symbol table** used by *all* phases.

```
char stream
 -> Scanner (lexical)    -> token stream
 -> Parser (syntax)      -> parse tree
 -> Semantic + IC gen    -> AST / IF
 -> Machine-indep. opt (optional)
 -> Target code gen      -> assembly
 -> Machine-specific opt (optional)
```

- **I Scanning** (char stream → **token stream**; ignores comments/whitespace) (smallest meaningful units; faster than char-by-char, simplifies later phases); recognizes *regular language* via DFA. **II Parsing** (tokens → **parse tree**): context-free structure (like syntax diagrams) via pushdown automata. **III Semantic analysis**: discovers meaning; *static* semantics = compile time (types); *dynamic* semantics = only at run time (array index out of bounds — **NOT** always catchable at compile time).
- **IV Intermediate form (IF)**: chosen for machine independence, easy optimization, compactness; resembles idealized machine (stack machine / infinite registers); may use several IFs. **V Optimization** (optional; misnomer — just "improve": faster/smaller). **VI Code gen**: assembly, relocatable machine code, or sometimes high-level language. **VII Machine-specific opt**: special instrs/addressing modes (x86-64 likes 8-byte words → smaller types can be *slower*).
- **Symbol table**: every identifier + what compiler knows about it; may be kept for debugger (flag).

Phases of Interpretation

- Same **frontend** (scanner → parser → semantic/IC → AST) but no backend: **tree-walk routines** take AST + program input and directly produce program output; symbol table still used.

Bootstrapping

- Compiler written in the language it compiles (C compiler is in C) → chicken-and-egg; bootstrapping solves it. Each component has 3 variables: **written in**, **input language**, **output language**.
- **Formula**: start with limited-feature bootstrap compiler → add features (while, exceptions) → compile new compiler with the old one → repeat. Analogous to OS *booting*.

```
Pascal bootstrap: #1 Pascal cmp in Pascal->P-code
#2 Pascal cmp in P-code->P-code  #3 P-code interp
in Pascal  #4 = #3 hand-translated to ASM
#5 Pascal cmp in Pascal->ASM  #6 Pascal cmp in
ASM->ASM.  Run #2 on #4 to compile #1; run #5
through #2 on #4 to get #6.
```

MODULE 2 — Syntax: RE, Scanners, CFGs, Parsers

- **Alphabet**: finite set of symbols. **String (lexeme)**: sequence of symbols from alphabet. **Language**: set of strings. **Syntax**: rules for which strings are legal-looking programs (**not** what a program does). **Semantics**: rules for meaning — what programs are legal and what they *do*.
- **RE** → recognizes *tokens* (can't say which tokens may sit next to each other); **Scanner/Lexer = DFA**. **CFG** → recognizes valid token *sequences*; **Parser = PDA** (pushdown automaton).

Regular Expressions

- **5 rules**: (1) empty string ε; (2) single character; (3) concatenation *rs*; (4) alternation *r*|*s*; (5) Kleene star *r** (0+). ε, |, * are *metacharacters*. REs are **greedy**: foobar321 is one token, 3.14159 is one token.

```
integer : digit digit*
decimal : digit* (. digit | digit .) digit*
exponent: (e|E) (+|-|ε) integer
real    : integer exponent
        | decimal (exponent | ε)
number  : integer | real
```

- **Test**: `007`, `.5`, `1E+10` valid; `-7` **NOT** valid (no sign rule for numbers).
- **Extended**: + 1+; ? 0/1; `{i,j}` i..j reps; `.` any char; `[aeiou]` set; ^ start; $ end. `sed 's/abc/cba/g'` (g = all, i = ignore case): `echo banana | sed 's/a/o/g'` → **bonono**. In awk, Java, JS, Go, Python.
- **Limits**: can't match nested/balanced parens `( x * (x + y) )` (needs recursion/memory); only regular languages. → CFG.

Scanner / Lexer (lexical analysis)

- Groups chars → **tokens**; gives stream of tokens to later stages; detects errors; lightens the costlier parsing; called *on demand* (parser asks for next token). Must accept/skip whitespace & comments. Tokens: numbers, identifiers, keywords, operators.
- **Machine parts**: input (the program text), position, character sets. **Functions**: **advance** (move ahead), **peek** (look at current char), **next** (gather next whole token).
- **Ad hoc**: handwritten code, one recognition routine per token type (lA1: `next()` is the driver). **Table-driven**: lookup table [state][char class] + engine; both = DFA. **lex** (1975) generates a lexer from REs.

```
state ws  letter digit  .  op | token
 1     6    2     3     4  5
 2          2     2           | id
 3                3     4    | num
 4                4           | num
 5                            | operator
 6     6    1     1     1  1
```

- **Scanner errors** (options): (1) discard the current invalid token; (2) skip ahead to a char that can start a valid token; (3) restart scanning; (4) leave it to later stages.

Context Free Grammars

- CFG = rules for a context-free language; "context free" = rules applied without conditions. Written in **BNF** (from Algol-60) / eBNF. More powerful than RE: rules can be *self-referential* (recursion), e.g. `nested : '(' nested ')'`
- **Terminal**: actual token (RE-recognized; parser doesn't care which id/number). **Non-terminal**: expands to terminals/non-terminals. **Production** = LHS → RHS; exactly *one* non-terminal on LHS, *no terminals* on LHS; RHS any mix. **Expansion**: LHS → RHS. | = disjunction.

```
expr -> id | number | - expr
     | ( expr ) | expr op expr
op   -> + | - | * | /      (ambiguous!)
```

- **Derivation**: sequential expansion of non-terminals from start to all-terminals. **Right-most**: always expand right-most NT; **Left-most**: expand left-most NT. **Parse tree**: graphical derivation (usually simplified to an **AST**).

```
slope * x + intercept (right-most):
expr => expr op expr => expr op id
 => expr + id => expr op expr + id
 => expr op id + id => expr * id + id
 => id * id + id
```

- **Ambiguous**: ≥2 distinct parse trees for a string (left-most vs right-most trees differ above). **Prove**: exhibit one string w/ two trees (`10 - 4 - 3`). Parser needs extra mechanism to choose. Infinite CFGs exist per language → pick an unambiguous one.

```
10 + 4 + 3 w/ expr -> expr op expr:
 expr( expr( expr 10, +, expr 4 ), +, expr 3 )
 expr( expr 10, +, expr( expr 4, +, expr 3 ) )
 two trees = ambiguous
```

```
expr   -> term | expr addop term
term   -> factor | term mulop factor
factor -> id | number | - factor
        | ( expr )
addop -> + | -    mulop -> * | /
```

- **Precedence**: ops lower in grammar (term/factor) bind tighter & sit deeper in tree (3+4*5: * under +). **Associativity**: left-recursive rule (expr addop term) → left assoc, (10-4)-3. Right-recursive → right assoc, 10-(4-3).

Parsers

- Parser = language recognizer & "the boss": calls lexer for tokens, builds parse tree piece by piece (**syntax-directed translation**). General algs **Earley** (1968), **CYK** (1961): dynamic programming, **O(n³)** — far too slow (~500k tokens ≈ 4 years); restrict CFG to get **O(n)**.
- **LL** (top-down / predictive): **L**eft-to-right, **L**eft-most derivation; builds tree root → down using current token + constant lookahead; **LL(1)** = one lookahead; pick production matching lookahead; mismatch = **syntax error**. Intuitive.
- **LR** (bottom-up / shift-reduce): **L**eft-to-right, **R**ight-most derivation *in reverse* (canonical); **shift** tokens as leaves until a production matches, then **reduce** into parent. More grammars are LR than LL; first widely used. Left-recursive grammars suit LR (`list_prefix → list_prefix , id`: LR but not LL); right-recursive shifts whole program before reducing.
- Variants: LR (SLR, LR(0), LR(1), **LALR**), LL (SLL, LL(1), full LL). Java = LALR; old GCC = LALR (Bison); modern GCC = hand-written recursive descent.
- **Left recursion** (`expr → expr addop term`) breaks LL: expr() would call expr() forever w/o consuming a token. Convert to right recursion `expr → term addop expr` (LL-able, but **breaks associativity**: 10-4-3 → 10-(4-3)) or Dragon Book form: `expr → term term_tail; term_tail → addop term term_tail | ε`
- **Recursive descent ↔ CFG**: Parser class, **one method per non-terminal**, plus a node class per non-terminal (parse-tree nodes); methods call other methods & consume tokens per production; go top → bottom of grammar.

```
Expr parseExpr() {  // expr: term addop expr | term
  Term t = parseTerm();
  Addop op = parseAddop();
  if (op == null) return new Expr(t);
  Expr e = parseExpr();
  e.append(new Expr(t, op, null));
  return e; }
```

- **Parser errors**: token ≠ expected → **SyntaxException**; recovery (general): panic mode — skip to sync token (; or }) & continue.
- **Tools**: **lex** (lexers), **yacc** = "Yet Another Compiler Compiler" (LALR), **bison** (GNU yacc), **ANTLR** (LL(*), recursive descent).

MODULE 3 — Names, Scope, Binding, Memory

- **Identifier**: char string representing something else. **Name**: identifier(s) (x.y.z). **Binding**: association (name ↔ memory address, value, type). **Scope**: textual part of program where a binding is active. **Environment**: set of bindings. Not all data is named (pointer targets).
- **Binding time**: language design (syntax, type system, semantics) → language implementation (CFG, I/O, numeric precision) → program development (algorithms, names) → **compile** (intra-module bindings, data layout) → **link** (inter-module, whole-program layout) → **load** (virtual addresses) → **run** (hardware addresses, variable values). **Static binding** (before run: more optimization, ~compiled) vs **dynamic** (at run: more flexibility, ~interpreted).
- **Lifetime / binding extent**: creation → destruction of binding. **Object** = any construct in memory (may differ from binding lifetime). Object that outlives all bindings = **garbage**; binding that outlives its object = **dangling reference**.

Memory Organization

- **3 allocations**: **Static/text/data** (absolute address kept whole run: machine code, globals, `static` vars, constants (strings, enums), small scalars in instructions); **Stack** (LIFO, follows calls/returns); **Heap** (arbitrary alloc/free; costly to manage). OS gives each process **virtual address** space (private; x86_64 Linux user space = low 256 TB; `pmap -x`). Stack grows toward low addr; heap toward high addr.
- **Stack frame** (per call/block; **not** the function name): locals, formal params, saved registers, scope info, return address, return value(s), prev. frame top. Code accesses data by **offset from frame pointer** (offsets known at compile time; absolute addresses not). Frame bracketed by frame ptr & stack ptr. Recursion grows stack; infinite recursion = **stack overflow**; **stack trace** = call stack dump. Why stack: natural for recursion, reuses space.
- **Calling sequence**: **caller** pushes actual params (calling sequence); **callee** *prologue* allocates locals; *epilogue* deallocates frame; caller reads return values just above stack top. Put as much as possible in prologue/epilogue (saves space). Ex: A→B→B→C→D: frames pushed in order; C's space reclaimed on return before D.
- **Heap**: data that must outlive a call. Java `new`; C `malloc` (**2 things**: pointer var on stack + object on heap; manual `free`; Java `new` same: reference on stack, object on heap); Go `make`/`new` (Go may put locals on heap). Allocation algorithms trade **time vs space**. Free-list alloc = O(n); buddy system O(lg n). **GC** pro: no manual free, prevents leaks/dangling refs. Con: runtime cost + language/compiler complexity.
- **Fragmentation**: *internal* = block bigger than object needs; *external* = freed gaps between live objects, hard to reuse.

Scope Rules

- Scope = block/module/class/subroutine/control statement. **Static scope ≠ static binding** (static binding = `const`/`final`; static-scoped langs still have dynamic bindings, e.g. changing list head). `static`: Java = class-level (not per instance); C = lifetime whole program even when name out of scope.
- **Referencing environment**: set of active bindings at a point; searched in order. Simple rules = one map; blocks = **stack of maps** (symbol table, built at compile time). Declaration stores name; assignment stores value (undeclared assign → error detected statically/dynamically, or acts as declaration in some langs).
- **Static (lexical)**: compiler resolves from program text; binding = nearest enclosing declaration (check local, then enclosing scopes); C, Go, Java, Scheme, JS. Block scope `{ }`: `{int temp=a; a=b; b=temp;}` temp freed after.
- **Nested subroutines** (Scheme, JS, Go): find non-local var via **static links** in frames: nesting depth of declaration d, of reference r; follow **k = r − d** links. Or **display** array: O(1). Why nest: private helpers, compact code, hide data in closures.
- **Dynamic**: by execution path (call chain); interpreted PLs w/o compile-time types (Bash); generally no closures → shallow binding. Impl: **A-list** (association list) as stack: push binding when seen at run time, pop on leaving scope, search top→bottom (slow access, fast calls); alt: table by identifier, each entry a stack of declarations (fast access, slow calls). Static compiler: stack of maps + frame code; dynamic interpreter: a-list.
- **Declaration order**: forward refs usually static semantic errors but needed for mutual recursion (Java allows for methods; JS hoisting).

Higher-Order Fns, Binding & Closures

- Fn assigned to var / passed / returned / built from data → which environment? **Deep binding**: environment fixed when fn *created* (early) — usually preferred; tends with static scope. **Shallow**: environment when fn *called* (late); tends with dynamic scope. **Closure** = fn + its referencing environment; implements deep binding (fn keeps *its own* env from creation — why `higher-order.js` doesn't see a later change); models objects, private data.

Meaning of Names

- **Shadowing**: same name → different objects by *scope*. **Overriding**: by *class* (subclass replaces inherited method; inheritance). **Aliasing**: same object, >1 name (`Names alias = obj;`). **Overloading**: same name → different objects by *type* (`f(int)` vs `f(double)`; operators + <<; confuse w/ `equals()` override). **Support**: C — no overloading, no namespaces (global unique names); Go — no overloading, package namespaces; Java — both.
- **Namespace**: set of names identifying objects; **module** = namespace unit; **package** = module.
- **Polymorphism**: one symbol, more than one type. **Subtype**: derived type inherits supertype properties; needs overriding; co/contravariance; Java classes, Go embedding + interfaces, C++ multiple inheritance, JS prototypal (+ class syntax). **Parametric/generics** (Java: **classes AND methods** can be parameterized): abstract type parameter (optionally upper/lower bounds), uniform behavior for any type: `public <T> void compute(T p)`; Go generics w/ constraints.
- **Macros**: token substitution, expanded at use (`gcc -E` shows). `#define SWAP(a,b) {int t=(a); (a=b); (b)=t;}` — name capture risk (which *t*?). Scheme: `syntax-rules` macros.
- **Static scope = lexical scope** (not runtime/heap/global).

GO (P1)

- **C**: systems lang (low level: chars, numbers, addresses, memory); no GC; `malloc` = library fn; segfaults. **Go** (Griesemer, Pike, Thompson, Google 2009; Andre's favorite): imperative, block-structured, **statically scoped**, strongly typed; compiles to machine code; **garbage collected**; arrays & maps built in; higher-order functions w/ closures & early binding; built-in concurrency (`go`) + channels.
- **Structure**: `package main` → `import` → types/funcs → `func main()`. **Quirks**: types come *after* names; named return vars; multiple returns; only `for` loop; **lowercase = package-private**, Capitalized = exported; `:=` infers type (vs =); semicolons auto-inserted; `defer`; errors returned as values; slices/maps.
- **Function vs Method**: function standalone `f(x)`; method has a **receiver** (param before name) binding it to a type, called `x.M()`. Pointer receiver `(p *T)` can modify original; value receiver gets a copy.

```
type Animal struct { Name string }
func (a Animal) Speak() string {
    return a.Name }  // receiver a
type Dog struct {
    Animal; Age int }  // embedding
d := Dog{Animal{"Rex"}, 3}
d.Speak()  // promoted method
```

- **OO = composition**: no classes/extends; only `package`, `interface`, `struct`. *Embed* a struct → fields & methods promoted. **Interfaces describe functionality, not identity**, satisfied implicitly. 1-method interface named method+"er" (`fmt.Stringer`).
- **Testing**: file `xyz_test.go`, funcs `TestXyz(t *testing.T)`, run `go test` (built in). Write *idiomatic* Go (Effective Go).
- **goroutine**: virtual thread, scheduled by Go *runtime* (not OS): `go f()`. **Race condition**: threads share memory; `b = 1 + b` = load, add, store — interleaving loses an update. **Atomic** = indivisible. Fixes: mutex, semaphore, monitor — Go: "*communicate by sharing messages, not data.*"
- **Channel**: statically typed, atomic, unbuffered or buffered pipe between goroutines. `var c chan int`; `c := make(chan int)`; send `c <- 1`; receive `b := <-c`. Unbuffered blocks until both sides ready; buffered `make(chan int, n)` blocks when full/empty. `close`, `range c`, `select`, `sync.WaitGroup`. Ex: sleep sort, parallel merge sort.

SCHEME (P2) / Functional

- **Functional PL**: outputs = math function of inputs; recursion, **no side effects** (side effect = modifies state outside local env / interacts w/ outside world). Pure = none allowed. Based on Church's **lambda calculus** (1930s; unnamed fns, 1 param). Needs: first-class & higher-order fns, polymorphism, lists, GC. **Innovations**: recursion, closures, lambda, lazy eval, macros (syntax-rules), tail-call opt, memoization, currying.
- **Scheme**: Lisp dialect (McCarthy, MIT 1958; *List Processing*). Dynamic types, **static scope**, higher-order fns, tiny syntax. Side-effect fns end in `!` (`set!`). **Program** = parenthesized symbolic expressions; expression always yields a value. Literal → itself; symbol → its value; `(f a b)` → call first w/ values of rest (**prefix**/**Polish**). Anything else = **special form** (`define if lambda quote`).
- **Prefix**: `(+ 1 2 3)`→6; `(* (+ 1 2) 4)`→12; `(lambda (x) (+ x 4))` = unnamed fn of x adding 4. Evaluate args (inner first), then apply.
- **Lists** = pairs + atoms; **homoiconic** (programs are lists too). `'(1 2 3)` = `(quote (1 2 3))` = don't evaluate. **car** = first, **cdr** = rest (names from IBM 701 registers); **cons** builds a pair. All equal: `'(a b c)`, `(list 'a 'b 'c)`, `(cons 'a (cons 'b (cons 'c '())))` = (a . (b . (c . ()))).
- `(cons 1 '(2 3))`→(1 2 3); `(car '(1 2 3))`→1; `(cdr '(1 2 3))`→(2 3); `(cdr '(1))`→(); `(cadr x)`=(car (cdr x)). `(cons 1 2)`→**(1 . 2)** improper/dotted pair (cdr not a list) — `(cons 1 '())`→(1) correct. `pair? null? list append length`.
- **Forms**: `(define x 5)`; `(define (f a) ...)`; `(lambda (x) ...)`; `(if test a b)`; `(cond (t1 e1) (else e2))`; `(let ((x 1)) ...)`; `map`/`filter`; closures.
- **Recursion**: base case (`null?`) + recurse on `cdr`, combine with `car`. Tail recursion = accumulator param (no stack growth).

```
(define (len l) (if (null? l) 0
   (+ 1 (len (cdr l)))))
(define (sum l) (if (null? l) 0
   (+ (car l) (sum (cdr l)))))
(len '(a b c)) -> 3  (sum '(1 2 3)) -> 6
```

JAVASCRIPT (P3) / Scripting

- **Scripting langs**: no precise def; ancestors = shell (sh, bash) + sed/awk. Traits: batch + interactive, economy of expression, no declarations, simple scoping, flexible dynamic typing, easy access to other programs, strong pattern matching/strings, high-level data types. Often contrasted with systems langs.
- **JS history**: Brendan Eich, Netscape, 1995, *10 days*; goal: Scheme's **functions** + Self's **object model** + Java's **syntax**. Standard = **ECMAScript** (ES5 strict mode; ES6/2015: `const let`, arrows; yearly; "ESNext"). Closer to functional than imperative: `(...nums) => nums.reduce((a,b)=>a+b, 0)` (arrow, spread, higher-order).
- **Good**: functions, community (NPM, Node, React...), objects/JSON, built-in regex, prototypal inheritance. **Bad**: == coercion, `eval`, `new`, `void`, hoisting, optional blocks. **Worse**: global vars, scope (no block scope w/o `let/const`), auto semicolon insertion, NaN, coercions.
- **OO**: **prototypal**: each object links (`__proto__`) to a prototype → chain until null; dynamic types. 3 styles: pseudoclassical/prototype, functional (closure), ES6 `class`; all give subtype polymorphism. **Encapsulation**: pure prototypes have no privacy; use **closure** or `#private` fields (ES22).
- **Modules**: CommonJS vs ES modules. **Async**: V8 single-threaded; **event loop** = non-blocking I/O via **callbacks** or **async/await** + promises. **Linting** = static analysis; **ESLint** (required for P3). **TypeScript**: statically typed, transpiles to JS. Lexical scope; `var` function-scoped & hoisted vs `let/const` block-scoped.

QUICK-CHECK (review-game facts)

- Syntax ≠ what a program does (that's **semantics**). Scanner = **RE/DFA**; Parser = **CFG/PDA**. RE **cannot** match nested parens. Scanner: chars→tokens, **ignores comments**; Parser: tokens→parse tree. Array bounds **not** always compile-time. Parsing = **context-free**. Ambiguity proof = one string, **two parse trees**. Bottom-up/shift-reduce = **LR**. Frame holds locals, params, return addr (**no function name**). Internal frag = block bigger than needed. malloc/new = pointer on **stack** + object on **heap**. Deep binding = bindings when **created**. Static = **lexical**. Shadow: same name→diff objects; alias: same object→diff names.